



Figure 1: xkcd.com comic picks on Linux

and mobile devices with less than 16 cores. According to him, the current kernel scheduler, CFS, fails to get the maximum out of the hardware in most common situations on a desktop.

As Kolivas writes in his FAQ page for BFS: "For years we've been doing our workloads on Linux to have more work than we had CPUs because we thought that the 'jobserver' were limited in their ability to utilise the CPUs effectively (so we did make `-j6` or more on a quad core machine, for example). This scheduler proves that the job servers weren't at fault at all, because make `-j4` on a quad core machine with BFS is faster than *any* choice of job numbers on CFS." The difference in speed is attributed to the inability of the mainline scheduler to keep the CPUs busy enough like BFS does.

Highlighting the design goals of BFS, he writes, "It was designed to be forward looking only, to make the most of lower spec machines, and not scale to massive hardware. That is, it is a desktop-orientated scheduler, with extremely low latencies for excellent interactivity by design rather than 'calculated', with rigid fairness, nice priority distribution and extreme scalability within normal load levels."

Thus BFS strives to produce extreme low latencies on machines with up to 16 CPUs. Anything more than this many CPUs will have exponentially less performance. So BFS may not perform well, or even perform worse, on huge machines with more number of CPUs. As the xkcd comic rightfully puts it: "But, who uses that?"

So, what other striking features does BFS have? Kolivas writes, "On top of the current scheduler design, it has a `SCHED_IDLEPrio` which actually does only schedule tasks when idle, and `SCHED_ISO` for unprivileged real-time performance. BFS does NOT implement `CGROUPS`. A desktop user should not need to know about `CGROUPS`, nor should they need to use them. BFS also does not have the feature of 'lots of tunables I don't understand'."

By removing most of the unnecessary features that a desktop computer doesn't require, it is the simplicity that works in favour of BFS.

How do I use BFS?

You may need to use one of the latest versions of the kernel and patch it with the appropriate BFS patch. Go and fetch the latest kernel (I choose version 2.6.31) from kernel.org:

```
cd /usr/src
wget -c http://www.kernel.org/pub/linux/kernel/v2.6/linux-2.6.31.tar.bz2
```

Then download the BFS patch for that kernel and apply the patch after extracting the kernel source:

```
wget -c http://ck.kolivas.org/patches/bfs/2.6.31-sched-bfs-220.patch
tar xjvf linux-2.6.31.tar.bz2
cd linux-2.6.31
patch -p1 < ../2.6.31-sched-bfs-220.patch
```

Then compile the kernel. If you are using Ubuntu, you can use the `make-kpkg` command to build the deb packages for *kernel-image* and *kernel-headers*. You may need to install *kernel-package* prior to this. It is as simple as typing a `sudo apt-get install kernel-package`. You can copy the current kernel configuration as follows:

```
cp -vi /boot/config-`uname -r` .config
```

Note that since Ubuntu kernels are built with *Debugging info* enabled, it makes the kernel modules much larger in size than they should be. So, if you're using the existing configuration, it's advisable to turn off the *Debugging info* mode. While you're at `make menuconfig`, go to 'Kernel hacking', and under 'Kernel debugging' turn off 'Compile the kernel with debug info'.

If you want to speed up the compilation process, you can enable parallel make by setting the `CONCURRENCY_LEVEL` environment variable. Set it to 1 plus the number of processor cores you have. For example, if you have a dual-core processor, use:

```
export CONCURRENCY_LEVEL=3
```

Change the kernel configuration:

```
make menuconfig
```

Additionally, if you're not compiling for the first time, run the following command for a clean build:

```
make-kpkg clean
```

Finally:

```
fakeroot make-kpkg --initrd --append-to-version=-bfskernel kernel-image
kernel-headers
```

This will take a long time and after this step, you will get two files, namely:

- `linux-headers-2.6.31-bfs211-bfskernel_2.6.31-bfs211-bfskernel-10.00.Custom_i386.deb`
- `linux-image-2.6.31-bfs211-bfskernel_2.6.31-bfs211-bfskernel-10.00.Custom_i386.deb`

Install them using the `dpkg -i <deb file>` command.